

**Problem 1:**

Carefully read and understand the posted solutions to the **midterm exam**. Identify **one problem from the list below** for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.).

- Q10 Chaotic Potential
- Q13 Greedy Cat Photographing
- Q14 DFA Impossibility
- Q15 SpongeBob vs Squidward
- Q16 Machine Disagreement

Copy or screenshot your solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't take the exam, review this solution by GPT 4o instead (pick one problem and explain the room for improvement).

10. You are observing a highly unstable process unfolding in a chaotic energy field. The process involves two types of particles: maize and blue. In each time unit, **one** of the following reactions must occur at random:

- **Fusion:** Two maize particles combine to form one blue particle.
- **Decay:** One blue particle destabilizes and transforms into one maize particle.

The process *halts* if no more reactions are possible.

- (a) Let  $b$  and  $m$  be the number of blue and maize particles. **Give a potential function  $s(b, m)$  that can be used to prove that this process eventually halts.**

$$s(b, m) = b + m$$

- (b) **Briefly (in 2-3 sentences) explain why your answer in Part a works.**

If fusion occurs then  $b+m \rightarrow (m-2)+(b+1) = b+m-1 < b+m$   
 If decay occurs then  $b+m \rightarrow b-1+m = b+m-1 < b+m$   
 Thus, decreases by fixed amount, and lowerbounded by zero since we can't have less than zero  $b, m$ .

My solution was deficient because I forgot that the amount of maize particles decreased by one in the decay stage.

---

I could have fixed with the potential function  $3b + 2m$ . Since if fusion then  $3b + 2m \rightarrow 3(b + 1) + 2(m - 1) = 3b + 2m - 1$  and if decay then  $3b + 2m \rightarrow 3(b - 1) + 2(m + 1) = 3b + 2m - 1$

## Problem 2: Understanding P, NP, and NP-hardness.

(a)

Prove that all efficiently decidable languages are efficiently verifiable. In other words, show that every language in P has an efficient verifier.

*Proof.*

Let  $L \in P$  thus there exists an efficient Turing Machine  $M$  that decides  $L$  in polynomial time.

Let  $V(x, c) = M(x)$ .

$V(x, c)$  is polynomial time since  $M(x)$  is.

If  $x \in L$  then  $M(x)$  accepts thus  $V(x, c)$  accepts.

If  $x \notin L$  then  $M(x)$  rejects thus  $V(x, c)$  rejects for all  $c$  since we ignore it. ■

(b)

Prove that every language in NP is decidable. In other words, show that every language in NP has a decider.

*Proof.*

Let  $L \in NP$  thus there exists an efficient verifier  $V$  that is bounded by  $p(|x|)$  a polynomial function.

Define  $M(x) = V(x, c)$  s.t. we loop over all strings  $c$  s.t.  $|c| \leq p(|x|)$  and accept if  $V(x, c)$  accepts or reject if  $V(x, c)$  rejects for all  $c$ .

If  $x \in L$  then there exists a  $c$  s.t.  $p(|x|) \geq |c|$ . Thus, our Turing machine  $M(x)$  will loop through all possible strings  $c$ . Thus, eventually we will find a  $c$  s.t.  $V(x, c)$  accepts thus  $M(x)$  accepts.

If  $x \notin L$  then there is no  $c$  s.t.  $p(|x|) \geq |c|$  s.t.  $V(x, c)$  will accept. Thus,  $M(x)$  will reject. Thus,  $M$  is a decider for  $L$ . Thus, NP is a subset of decidable languages. ■

(c)

Prove or disprove: All NP-hard languages are decidable.

(A disproof would consist of an NP-hard language that is also undecidable.)

*Proof.*

False consider  $L_{Halt}$ . This is NP-Hard by proven on discussion worksheet complexity and undecidable. ■

### Problem 3: Verifying Y's verifiers.

Recall that a *multiset*  $S$  is just like a set, but it can have duplicate elements. A sub-multiset  $S^* \subseteq S$  also can have duplicate elements, as long as it does not have more copies of a particular element than  $S$  does.

$\text{SUBSETSUM} = \{(S, k) : S \text{ is a multiset of integers } \geq 0, \text{ and } \exists S' \subseteq S \text{ such that } \sum_{s \in S'} s = k\}.$

(a)

---

**Require:** A multiset of nonnegative integers  $S = \{s_1, s_2, \dots, s_n\}$ , an integer  $k$ , and a multiset of nonnegative integers  $C = \{c_1, c_2, \dots, c_m\}$

```

1: function VERIFIERA( $S, k, C$ )
2:   if  $m > n$  then reject           ▷  $C$  can't be a submultiset of  $S$  in this case
3:    $sum \leftarrow 0$ 
4:   for  $c \in C$  do                     ▷ iterate over elements (integers) in  $C$ 
5:     if  $c \notin S$  then reject
6:     else  $sum \leftarrow sum + c$ 
7:   if  $sum = k$  then accept
8:   reject

```

---

**Correctness:** For correctness,

$$\begin{aligned}
 (S, k) \in \text{SUBSETSUM} &\iff \text{there exists a submultiset } C \subseteq S \text{ s.t. } \sum_{c \in C} c = k \\
 &\iff \text{VERIFIERA}(S, k, C) \text{ accepts}
 \end{aligned}$$

**Runtime:** After ruling out the possibility that  $m > n$  in line 2, the for loop will iterate over at most  $n$  elements, while other lines take constant time, the overall runtime is  $O(n)$ , which is polynomial with respect to the input size.

Explain why VERIFIERA does not show that  $\text{SUBSETSUM} \in \text{NP}$ . You may use an example to support your reasoning. You do not have to fix the verifier, though it is a good exercise.

*Proof.*

VERIFIERA does not work since it does not account for if  $C$  has more elements of one type than  $S$ . For example if  $S = \{1, 100, 100, 100, 100\}$ ,  $k = 5$ , and  $C = \{1, 1, 1, 1, 1\}$ . Then  $V(S, k, C)$  would accept since  $1 \in S$ . Thus,  $sum \leftarrow 5 = k$ , but  $S$  cannot sum to 5. ■

(b)

---

**Require:** A multiset of nonnegative integers  $S = \{s_1, s_2, \dots, s_n\}$ , an integer  $k$ , and a collection of nonnegative integers multisets  $C = \{C_1, C_2, \dots, C_m\}$

---

```

1: function VERIFIERB( $S, k, C$ )
2:   for  $C_i \in C$  do                                ▷ Iterate over multisets in  $C$ 
3:     Confirm that  $C_i \subseteq S$ 
4:     if  $\sum_{c \in C_i} c = k$  then accept
5:   reject

```

---

**Correctness:** For correctness,

$$\begin{aligned}
 (S, k) \in \text{SUBSETSUM} &\iff \text{there exists a submultiset } C_i \subseteq S \text{ s.t. } \sum_{c \in C_i} c = k \\
 &\iff \text{there exists a collection } C \text{ that contains } C_i \\
 &\iff \text{VERIFIERB}(S, k, C) \text{ accepts}
 \end{aligned}$$

**Runtime:** In line 4, since  $C_i$  is confirmed to be a submultiset of  $S$ ,  $|C_i| \leq n$  and hence summing all items takes  $O(n)$ . Since there are  $m$  sets in  $C$ , the overall runtime is  $O(mn)$ , which is polynomial with respect to the input size.

Explain why VERIFIERB does not show that SUBSETSUM  $\in$  NP. You may use an example to support your reasoning. You do not have to fix the verifier, though it is a good exercise.

*Proof.*

Consider  $C = \{C_1, C_2\}$  s.t.  $C_1 \subset S$  whose elements sum to  $k$ , and  $C_2$  s.t.  $|C_2| = |S|^4$ . Thus, even though the certificate is correct  $|C_2|$  is not a polynomial in  $|S|$ . ■

(c)

---

**Require:** A multiset of nonnegative integers  $S = \{s_1, s_2, \dots, s_n\}$ , an integer  $k$ , and an array of bits  $C[1, \dots, n]$

---

```

1: function VERIFIERC( $S, k, C$ )
2:    $sum \leftarrow 0$ 
3:   for  $i = 1, \dots, n$  do
4:     if  $C[i] = 1$  then  $sum \leftarrow sum + s_i$ 
5:   if  $sum = k$  then accept
6:   reject

```

---

Y claims that Lambda's verifier is incorrect with the following counterexample:

Consider  $S = \{1, 1, 2, 5\}$ ,  $k = 7$ , and  $C = [0, 1, 0, 1]$ . Clearly,  $S \in \text{SUBSETSUM}$ , but  $1 + 5 = 6 \neq 7$  so the verifier rejects. Therefore, VERIFIERC must be incorrect!

Explain why Y's claim is unfounded.

This is unfounded since there exists  $C = [0, 0, 1, 1]$  s.t.  $V(S, 7, C)$  accepts. We cannot throw out a verifier if one  $C$  does not work we have to check all  $C$  of a certain size.

### Problem 4: Understanding poly-time mapping reductions.

(a)

Prove or disprove: if  $A \leq_p B$ , then  $A \leq_T B$ .

(A disproof would consist of a pair of languages such that  $A \leq_p B$  but  $A \not\leq_T B$ .)

*Proof.*

Assume  $A \leq_p B$  then this means there exists a efficient computable  $f : \Sigma^* \rightarrow \Sigma^*$  s.t.  
 $x \in A \iff f(x) \in B$ .

Let  $M_B$  be an oracle that decides  $B$ .

Define:  $M_A(x) = M_B(f(x))$ .

$M_A$  halts on any input because  $f$  is efficiently computable, and  $M_B$  halts on every input.

Therefore,

$$\begin{aligned} x \in A &\iff f(x) \in B \\ &\iff M_B(f(x)) \text{ accepts} \\ &\iff M_A(x) \text{ accepts} \end{aligned}$$

Thus,  $M_A$  decides  $A$ . Thus,  $A \leq_T B$ . ■

(b)

Prove or disprove: if  $A \leq_T B$ , then  $A \leq_p B$ .

(A disproof would consist of a pair of languages such that  $A \leq_T B$  but  $A \not\leq_p B$ .)

*Proof.*

Let  $A$  be SAT.

Let  $B = \{x0^{2^{|x|}} \mid x \in A\}$ , Every string  $x$  in SAT is padded with  $2^{|x|}$  zeros.

Let  $M_B$  be an oracle that decides  $B$ .

Define  $M_A(x)$  s.t. we create a new string  $y = x0^{2^{|x|}}$ . Then return  $M_B(y)$ .

$M_A$  halts since  $M_B$  halts.

$$\begin{aligned} x \in A &\iff y = x0^{2^{|x|}} \in B \\ &\iff M_B(y) \text{ accepts} \\ &\iff M_A(x) \text{ accepts.} \end{aligned}$$

Thus,  $A \leq_T B$

Assume for contradiction that  $A \leq_p B$  then there exists an efficiently computable  $f$  s.t.  $x \in A \iff f(x) \in B$ .

Thus, there is some polynomial  $q$  that bounds  $f$ . Since writing an output takes at least one step this implies that the machines cannot output more than  $q(|x|)$  symbols. Thus,  $|f(x)| \leq p(|x|)$ .

Therefore, if  $x \in A$  then  $f(x) \in B$ . This means that there exists some  $z$  s.t.  $f(x) = z0^{2^{|z|}}$ .  
Therefore,  $|f(x)| = |z| + 2^{|z|} \leq p(|x|)$  ■

(c)

Prove that PTMRs are *transitive*: if  $A \leq_p B$  and  $B \leq_p C$ , then  $A \leq_p C$ .

*Proof.*

Assume  $A \leq_p B$  and  $B \leq_p C$ .

Thus, there exists efficiently computable  $f, g$  s.t.  $x \in A \iff f(x) \in B$  and  $y \in B \iff g(y) \in C$ .

Thus,  $g \circ f$  is efficient computable.

Thus,  $x \in A \iff f(x) \in B \iff g(f(x)) \in C$ .

Therefore,  $A \leq_p C$  ■

(d)

The languages  $\Sigma^*$  and  $\emptyset$  are called the *trivial* languages, and all other languages are *non-trivial*. Prove that if  $P = NP$ , then *every* non-trivial language is NP-hard.

Be sure to state where your proof relies on non-triviality (because the trivial languages are *not* NP-hard, regardless of whether  $P = NP$ !).

*Proof.*

Assume  $P = NP$ . Let  $L$  be a non-trivial language.

Let  $A \in NP = P$ . Thus, there is an efficient Turing machine  $M_A$  that decides  $A$ .

Since  $L$  is non-trivial this means there exists distinct  $x, y \in \Sigma^*$  s.t.  $x \in L$  and  $y \notin L$ .

Thus define  $f(a) = x$  if  $M_A(a)$  accepts else  $y$ .

Thus,

$$\begin{aligned} a \in A &\iff M_A(a) \text{ accepts} \\ &\iff f(a) = x \\ &\iff f(a) \in L. \end{aligned}$$

$f$  is efficient computable since  $M_A$  is, and is correct by the proof above and the fact that  $M_A$  is correct and  $x \neq y$ .

Thus,  $A \leq_p L$  for all  $A \in \text{NP}$ .

Thus,  $L$  is NP-Hard. ■